```
  maxSize ← maxElements

  hashTable ← new
HashTable(maxElements)
  elements ← new PriorityQueue()
                                                #1
```

#1 This time we need to create a priority queue. On the other hand, we don’t need the pointer to the tail anymore. We assume that elements with lower values for priorities are towards the top of the queue (like, for instance, in a min-heap).

It might be a heap, a Fibonacci heap, or any other implementation – we will stick with the abstract data structure in the code, leaving to users the task of thinking which implementation works best for them, depending on the programming language they use and, of course, the context they will use the cache[25]. For the performance analysis, though, we will consider a heap because it gives us reasonable enough performance guarantees on all operations, combined with a clean, simple implementation.

Take a look at figure 7.11 to see how this changes our cache internals. Now we have only two things to change in our implementation to switch from an LRU to an LFU:

1. The eviction policy (all the logic about choosing which element to remove)
2. The data structure used to store elements.

It’s time to look at some more code: we highlighted the differences with the corresponding LRU’s methods to make it